

AN IMPROVED ROM COMPRESSION TECHNIQUE FOR DIRECT DIGITAL FREQUENCY SYNTHESIZERS

Mohamed M. El Said and M. I. Elmasry

VLSI Research Group, University of Waterloo
Waterloo, Ontario, Canada N2L 3G1

ABSTRACT

In this paper a comparison between the famous ROM compression techniques, for ROM-based direct digital frequency synthesizers, was performed. Inaccuracies of previous comparisons are pointed out. An improved ROM compression technique based on the Taylor-series linear interpolation method, developed by Bellaouar *et al.* [2], is presented. Simulation results show that compared to previously reported compression techniques, this improvement can achieve considerable reductions in the ROM size.

1. INTRODUCTION

The most common direct digital frequency synthesizer (DDFS) architecture is the Tierney, Rader, Gold architecture [1]. In this architecture a ROM is used as a lookup table to convert the digital phase to output digital sine samples. For such architectures the size of the ROM can represent a major bottle neck for high-speed applications [3]. Also the ROM is the major source of power consumption in a DDFS [2]. Thus compression algorithms to reduce the actual size of the ROM are essential for high-speed and low-power DDFS applications.

In this paper a comparison between the famous ROM compression techniques was performed. An improvement to the latest compression technique introduced by Bellaouar *et al.* [2] is presented. Comparison of this improved technique to the previous ROM compression techniques shows that it can achieve a considerable savings in the required ROM capacity, at the expense of a slight loss in performance and an increase in the required hardware overhead.

2. DDFS FUNDAMENTALS

In its simplest form, a direct digital synthesizer can be implemented from a precision reference clock, an N-bit address counter, a read only memory (ROM), and a digital to analog converter (DAC). Each number generated by the counter is used to select a value from the ROM that corresponds to a sample of a sinusoid. The result is then con-

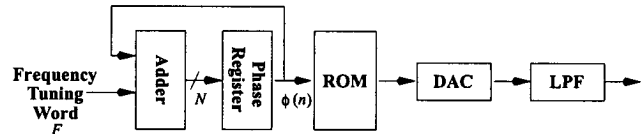


Fig. 1. Conventional DDFS architecture

verted to analog form and then filtered to suppress high frequency components. If the counter increments its output by a programmable step F , then the counter addresses fewer (evenly spaced) points of one cycle of the sinusoid and the output frequency would be higher. Such a counter can be implemented as an accumulator. The resulting architecture is shown in Fig.1 [1]. The 2^N words of the accumulator are mapped into phase values, $\theta(n)$, such that

$$\theta(n) = \frac{2\pi\phi(n)}{2^N} \quad (1)$$

where $\phi(n)$ represents the contents of the accumulator at time $t = n/f_{clk}$.

To keep the size of the ROM within practical limits only the P most significant bits of the accumulator output are used to provide phase information to the ROM. However, this comes at the expense of distorting the synthesized sine wave. The ROM size can be further minimized by employing ROM compression techniques as will be discussed in the following section. However, a nonlinear distortion is introduced.

3. ROM STORAGE COMPRESSION

3.1. Exploitation of Sine Function Symmetry

The most elementary method of sine storage compression is to exploit the symmetry of the sine function about π and $\pi/2$. By properly inverting the phase and the amplitude of the sine function, look-up table samples need only be stored for phase values between 0 and $\pi/2$.

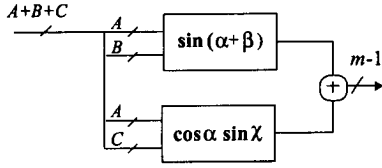


Fig. 2. Sunderland's architecture.

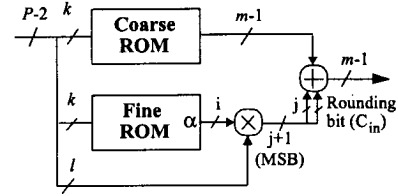


Fig. 3. Bellaouar's architecture.

3.2. Compression of Quarter-wave Sine Function

One simple technique of compressing data even further was first introduced by Hutchison [5]. It is based on breaking up the ROM into 2 parts: a coarse ROM and a fine ROM. The coarse ROM stores a coarse approximation of the sine wave by storing only 2^K samples where K are the MSB's of the phase word. The fine ROM stores the difference between the true value and the value of the sine at the 2^K points. Since the values stored in the fine ROM are relatively smaller than that stored in the coarse ROM, we find that the dynamic range of the fine ROM is reduced, and this is the major source of the gain in compression.

An improvement introduced by Sunderland [6] achieved substantial ROM compression. This architecture which is shown in Fig.2, provided a reduction in the ROM storage requirements by replacing the storage requirements of one large ROM of size 2^{A+B+C} words with two smaller ROM's of sizes 2^{A+B} words and 2^{A+C} words, whose outputs are added together to reconstruct the sine function. The assignment of look-up table samples is based on trigonometric approximations of the sine wave.

Using the same architecture as that of Sunderland's, Nicholas achieved significant improvements by using computer optimization for choosing the samples to be stored in the ROM's instead of trigonometric approximations[4].

In [2], further ROM reduction was achieved using a Taylor-series based linear interpolation between consecutive points stored in a coarse ROM as shown in Fig. 4(a). This can be explained by writing the sine function in every interval (θ_i, θ_{i+1}) as follows

$$\sin \theta = \sin \theta_i + \alpha(\phi - \phi_i) + \delta_{sin} \quad (2)$$

where $\theta \in [\theta_i, \theta_{i+1}]$ and $\theta = 2\pi\phi/2^P$. θ_i and θ_{i+1} are two successive stored phases. The " $\phi - \phi_i$ " term represents the LSB's of the phase accumulator ϕ , while ϕ_i represents the MSB address of the ROM. The interpolation coefficient α given by $2\pi\cos(\theta_i)/2^P$, is also stored in the ROM. But since this coefficient value is small, only a few MSB's are stored. Fig. 3 shows a simplified block diagram for this architecture. Compared to the previously mentioned architectures, an extra multiplier is required. However, for a sufficient compression level this overhead can be justified. In equation (2), δ_{sin} is the residual of the Taylor expansion

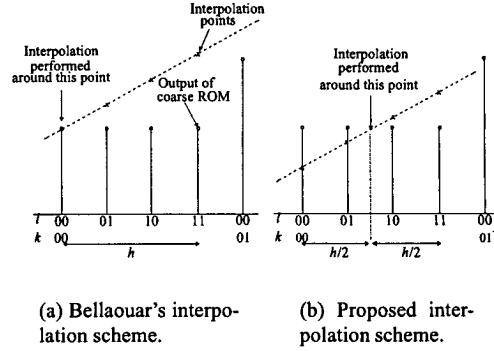


Fig. 4. Interpolation schemes based on the Taylor-series expansion.

of the sine function. This residual can be shown to have a maximum value of $h^2/2$ where h is the maximum spacing between the interpolation points and θ_i . As an improvement, the maximum difference $|(\theta - \theta_i)|_{max}$ can be reduced by half by performing the interpolation about a point in the middle of the interpolation range, instead of performing a forward interpolation around θ_i as shown in Fig. 4. Thus $|\delta_{sin}|_{max}$ is reduced to $h^2/8$. This gain in performance can be traded for a reduction in the ROM size. In other words, the same performance can be achieved by reducing the words stored in the coarse and fine ROM's by half (i.e. reducing k to $k-1$), and doubling the number of interpolation points (i.e. increasing l to $l+1$). This improvement can be achieved at the expense of small hardware modifications. In order to be able to perform interpolation around a point in the middle of the interpolation range, an offset of $-2^{l-1} + \frac{1}{2}\text{LSB}$ has to be added to the l bits representing $\phi - \phi_i$. This can be easily performed by complementing the MSB of the l bits representing $\phi - \phi_i$ and appending a $\frac{1}{2}\text{LSB}$. The multiplier has to be modified to accommodate 2's complement multiplication, which can be easily done using a Baugh-Wooley multiplier [8]. Fig. 5 depicts the resulting architecture. However, a small degradation in spectral purity is expected. This can be explained by realizing that increasing the number of interpolation bits by 1 increases $(\theta - \theta_i)_{max}$ from $2\pi(2^l - 1)/2^P$ to $2\pi(2^{l+1} - 1)/2^{P+1}$, i.e. an increase by $2\pi/2^{P+1}$ results. Simulation results in

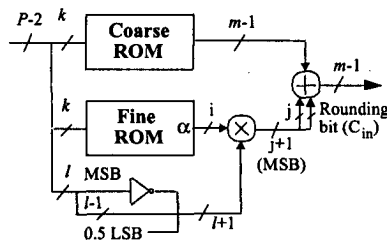


Fig. 5. Proposed architecture.

section 5 will show that this increase will have a negligible effect on the spectral purity.

3.3. Sine-Phase Difference Algorithm

For all the aforementioned techniques, further compression of the storage required for the quarter wave sine function can be obtained by storing the function $\sin(2\pi\phi/2^N) - \phi/2^N$ in the look-up table instead of $\sin(2\pi\phi/2^N)$ [4]. Because $\max[\sin(2\pi\phi/2^N) - \phi/2^N] = 0.21 \max[\sin(2\pi\phi/2^N)]$, this saves 2b of amplitude in the storage of the sine function. The penalty for this storage reduction is the introduction of an extra adder at the output of the look-up table to perform the operation $[\sin(2\pi\phi/2^N) - \phi/2^N] + \phi/2^N$.

4. SIMULATION CONDITIONS

Computer simulations were used to simulate the effects of the memory compression and algorithmic techniques on the output spectrum of the considered DDFS architectures. For each architecture the accumulator size (N) was set to 20 bits where the 12 MSB's (P) were used for addressing the ROM. Similar settings were used for the chip fabricated in [2], and are used in this paper as a common benchmark to compare the different DDFS architectures. Furthermore, in order to obtain a fair comparison between all architectures, results should be based on worst case spurious response. This requires simulating the architecture for each possible frequency word and accumulator initial condition, which makes the simulation time prohibitively long. Simulations in [2] neglected this fact which led to an overestimation of the compression capabilities of their technique. However, a simple hardware modification to the accumulator, as proposed in [4], can force its output to have a maximal numerical period for all values of F , i.e. all values of the phase accumulator output sequence are generated before any of the values are repeated. This modification has the additional advantage of improving the worst-case spur amplitude by 3.922 dB. Therefore a very consistent and predictable DDFS performance results for all values of F .

Table 1. Effect of phase segmentation on the output spectral purity for Bellaouar's architecture and the proposed architecture.

(k, l)	Spectral purity for Bellaouar's architecture (dBc)	Spectral purity for the proposed architecture (dBc)
(5,5)	-72.245	-72.245
(4,6)	-63.15	-72.1
(3,6)	-51.4	-59.9

5. SIMULATION RESULTS

According to [4] for $N=20$ and $P=12$, the maximum theoretical spur level is -72.247 dBc. This represents the best performance that can be achieved by any of the discussed architectures.

Table (1) shows the effect of varying the phase address segmentation on the output spectral purity for Bellaouar's architecture and the proposed architecture. The 2nd and 3rd entries in the table indicate that the proposed improvement can achieve a 50% reduction in the stored ROM words at the expense of a slight degradation in spectral purity.

Similar simulations were performed for Nicholas's and Sunderland's architectures. This resulted in an optimal phase segmentation of $(A, B, C) = (3, 3, 4)$ for Nicholas's architecture and $(A, B, C) = (3, 4, 3)$ for Sunderland's architecture. In [7] the phase segmentation for Nicholas's architecture was unnecessarily taken as (4,3,3), which increases the ROM size by 50%, and underestimates the compression capabilities of Nicholas's technique compared to other techniques.

Also the effects of various ROM output quantization were simulated to determine the minimum word-lengths required to achieve the required performance in terms of spectral purity. Table 2 presents the final simulation results. Results clearly indicate the compression advantage of the proposed improvement where it approximately achieves a 40% reduction in the ROM size over that of Bellaouar's. The hardware overhead increases from a 5x5 multiplier for Bellaouar's architecture to an 8x7 multiplier for the proposed architecture. For high speed designs this tradeoff can be advantageous since a multiplier's speed can be readily increased by pipelining, while such is not the case for the ROM. For low-power applications, this tradeoff has to be justified. For large ROM's, the multiplier consumes a small fraction of the power budget and thus power reduction through ROM size reduction can be very effective. However, for small ROM's the increase in the power consumption of hardware overhead can overwhelm any ROM power savings.

Table 2. Comparison of ROM compression techniques for an $N=20$ and $P=12$ case.

Method	Needed ROM	Total compression ratio	Spectral Purity (dBc)	Comments
Uncompressed memory ROM	$2^{12} \times 8$ bits	1:1	-72.245	reference
1/4 sine-wave symmetry	$2^{10} \times 8$ bits	4:1	-72.245	-
Hutchison's technique*	$2^7 \times 6$ bits $2^{10} \times 2$ bits	11.64:1	-72.245	2 adders required
Sunderland's technique*	$2^7 \times 8$ bits $2^6 \times 3$ bits	27:1	-72.245	(3,4,3) phase segmentation 2 adders required
Nicholas's technique*	$2^6 \times 7$ bits $2^6 \times 3$ bits	51.2:1	-72.245	(3,3,4) phase segmentation, 2 adders required
Bellaouar's technique*	$2^5 \times 8$ bits $2^5 \times 5$ bits	78.8:1	-72.245	(5,5) phase segmentation, 1 5x5-multiplier and 2 adders required
Proposed technique*	$2^4 \times 8$ bits $2^4 \times 8$ bits	128.8:1	-72.1	(4,6) phase segmentation, 1 8x7-multiplier and 2 adders required

*Sine-phase difference algorithm is used

6. CONCLUSION

A comparison of the famous ROM compression techniques has been presented, where the inaccuracies of previous comparisons were pointed out. It was shown that through proper phase word segmentation, Nicholas's compression technique can achieve a higher level of compression than that reported in [2] and [7]. An improved ROM compression technique, based on that in [2], was proposed. It has been demonstrated that it can achieve considerable ROM compression at the expense of a very slight decrease in performance and an increase in hardware overhead. This hardware overhead can be justified for high-speed applications where the large ROM sizes can represent a bottleneck. For low-power applications, the reduction in power through ROM size reduction has to be weighed against the increase in overhead power consumption, so as to determine the most suitable compression technique.

7. REFERENCES

- [1] J. Tierney *et al.*, "A digital frequency synthesizer," *IEEE Trans. Audio Electroacoustics.*, vol. AU-19, pp. 48-57, 1971.
- [2] A. Bellaouar *et al.*, "Low-power direct digital frequency synthesis for wireless communications," *IEEE J. Solid-State Circuits*, vol. 35, no. 3, March 2000, pp. 385-390.
- [3] B. Goldberg, *Digital Frequency Synthesis Demystified*. Eagle Rock, VA: LLH Technology Publishing, 1999.
- [4] H. T. Nicholas, *et al.*, "The optimization of direct digital frequency synthesizer performance in the presence of finite word length effects," *Proc. 42nd Annual Frequency Control Symp. USERACOM* (Ft. Monmouth, NJ), May 1988, pp. 357-363.
- [5] B. H. Hutchison, Jr., *Frequency Synthesis and Applications*. New York: IEEE Press, 1975.
- [6] D. A. Sunderland *et al.*, "CMOS/SOS frequency synthesizer LSI circuit for spread spectrum communications," *IEEE J. Solid-State Circuits*, vol. SC-19, August 1984, pp. 497-505.
- [7] J. Vankka *et al.*, "A direct digital synthesizer with an on-chip D/A-converter," *IEEE J. Solid-State Circuits*, vol. 33, no. 2, February 1998, pp. 218-227.
- [8] A. Bellaouar, and M. I. Elmasry, *Low-Power Digital VLSI Design: Circuits and Systems*. : Kluwer Academic Publishers, 1995.